txt2tex Reference

Write math like you're at a whiteboard. Get LaTeX you can hand in.

https://github.com/jmf-pobox/txt2tex

Document Structure

`=== Title ===`	`\section *{Title}`
`* Solution N **`	Bold heading with spacing
`(a) Text`	Part label
`TEXT: paragraph`	Prose block (smart formula detection)
`LATEX: \cmd`	Raw LaTeX passthrough
`PAGEBREAK:`	Page break

`TITLE: ...`	Document title
`AUTHOR: ...`	Author name
`DATE: ...`	Date string
`BIBLIOGRAPHY: f.bib`	BibTeX file
`[cite key]`	`\citep {key}`

Identifier Decoration

`count'`	*count*′ (after-state)
`in?`	*in*? (input)
`out!`	*out*! (output)
`x?'`	*x*?′ (runs may combine)
`'sunk'`	'sunk' (string literal)

Decoration attaches to any identifier. Reserved words (`theta`, `defs`, `hide`, `project`, `Delta`, `Xi`) cannot be decorated.

Logic

`p land q`	$p \land q$
`p lor q`	$p \lor q$
`lnot p`	$\neg p$
`p => q`	$p \Rightarrow q$
`p <=> q`	$p \Leftrightarrow q$
`forall x : S | P`	$\forall x : S \bullet P$
`exists x : S | P`	$\exists x : S \bullet P$
`exists_1 x : S | P`	$\exists_1 x : S \bullet P$
`lambda x : S | E`	$\lambda x : S \bullet E$
`mu x : S | P`	$\mu x : S \bullet P$

Note: Use `land`, `lor`, `lnot`. English `and`/`or`/`not` are not supported.

Sets & Types

`x elem S`	$x \in S$
`x notin S`	$x \notin S$
`A subset B`	$A \subseteq B$
`A union B`	$A \cup B$
`A inter B`	$A \cap B$
`A setminus B`	$A \setminus B$
`power S`	$\mathbb{P}\, S$
`F S`	$\mathbb{F}\, S$
`{x : S | P}`	Set comprehension
`A cross B`	$A \times B$
`N / Z / N1`	$\mathbb{N}$ / $\mathbb{Z}$ / $\mathbb{N}_1$
`n..m`	$n \mathinner{..} m$

Relations

`A <-> B`	$A \leftrightarrow B$
`x |-> y`	$(x \mapsto y)$
`dom R / ran R`	$\mathrm{dom}\, R$ / $\mathrm{ran}\, R$
`A dres R`	$A \lhd R$
`A dsub R`	$A \lhd\!\!\!\cdot\, R$
`R rres B`	$R \rhd B$
`R rsub B`	$R \rhd\!\!\!\cdot\, B$
`R oplus S`	$R \oplus S$ (override)
`R o9 S`	$R \fatsemi S$ (forward comp.)
`R comp S`	$R \fatsemi S$ (backward comp.)

Functions

`A pfun B`	$A \pfun B$
`A fun B`	$A \to B$ (total)
`A pinj B`	$A \pinj B$
`A inj B`	$A \inj B$
`A surj B`	$A \surj B$
`A bij B`	$A \bij B$
`A ffun B`	$A \ffun B$ (finite)
`f(x)`	Function application

Sequences

`<> / <a, b>`	$\langle\rangle$ / $\langle a, b\rangle$
`s ^ t`	$s \frown t$ (concatenation)
`seq S / seq1 S`	$\mathrm{seq}\, S$ / $\mathrm{seq}_1\, S$
`# s`	$\#s$ (length)

Z Notation Blocks

`given A, B`	Given types: $[A, B]$
`T ::= a | b`	Free type
`name == expr`	Abbreviation

`axdef` / `schema Name` / `gendef [X]`

`declarations`	Variable declarations
`where`	Separator
`predicates`	Constraining predicates
`end`	Close the block

Schema Inclusion & Calculus

`Delta Counter`	$\Delta \mathit{Counter}$ (before/after)
`Xi Card`	$\Xi \mathit{Card}$ (read-only)
`SName`	bare inclusion
`theta S`	θS (state binding)
`theta S'`	$\theta S'$ (after-state)

`Name defs RHS` — horizontal definition:

`Name defs Schema`	$Name \mathrel{\widehat=} Schema$
`N defs Delta C`	$N \mathrel{\widehat=} \Delta C$
`N defs [d : T | P]`	inline schema text
`N[X] defs G[X]`	generic form

`defs` RHS operators (schema calculus):

`S[old/new]`	rename component
`S[a/b, c/d]`	multiple renames
`S ; T`	$S \fatsemi T$ (composition)
`S » T`	$S \gg T$ (piping)
`S hide (x, y)`	$S \setminus (x, y)$
`S project T`	$S \upharpoonright T$

Proof Tree Syntax

indent	Premises indented under conclusion
`[rule]`	Justification label
`[rule [n]]`	Discharge assumption n
`[n] expr`	Boxed assumption labelled n
`::`	Sibling premises

Rules: `land intro`, `land elim 1/2`, `lor intro 1/2`, `lor elim`, `=> intro/elim`, `<=> intro/elim`, `lnot intro/elim`, `forall intro/elim`, `exists intro/elim`.

Usage

`txt2tex f.txt`	→ f.tex + f.pdf
`txt2tex f.txt --tex-only`	→ f.tex only
`txt2tex -i`	Interactive REPL
`txt2tex --check-env`	Verify dependencies

txt2tex Proofs — By Example

Each example shows what you type and what txt2tex renders.

How Proof Trees Work

In txt2tex, proof trees are written **bottom-up**: the conclusion is the first line, and its premises are indented below it. Think of it as reading the inference rule from conclusion to justification:

conclusion	First line (least indented)
premise	Indented = supports the line above
`[rule]`	Justification in brackets
`[n] expr`	Boxed assumption (labelled n)
`[rule [n]]`	Discharge assumption n
`::`	Marks sibling premises

Deeper indentation = deeper in the tree. Two premises at the same indent level are siblings that jointly support their parent.

Truth Table Prop. Logic

You write:

```
TRUTH TABLE:
p | q | p => q
T | T | T
T | F | F
F | T | T
F | F | T
```

You get:

p	q	$p \Rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Equivalence Chain Prop. Logic

You write:

```
EQUIV:
lnot (p land q)
lnot p lor lnot q  [De Morgan]
```

You get:

$$\begin{aligned} & \neg\,(p \wedge q) \\ \Leftrightarrow\ & \neg\,p \vee \neg\,q \qquad \text{[De Morgan]} \end{aligned}$$

`EQUIV:` joins steps with $\Leftrightarrow$. Use for *predicate* equivalences.

Modus Ponens Proof Trees

You write:

```
PROOF:
  q [=> elim]
    p => q
    p
```

You get:

$$\frac{p \Rightarrow q \quad p}{q}\ \Rightarrow\text{-elim}$$

$\wedge$-intro Proof Trees

You write:

```
PROOF:
  p land q [land intro]
    p
    q
```

You get:

$$\frac{p \quad q}{p \wedge q}\ \wedge\text{-intro}$$

$\Rightarrow$-intro with discharge Proof Trees

You write:

```
PROOF:
  p => p [=> intro [1]]
    [1] p
```

You get:

$$\frac{\ulcorner p \urcorner^{[1]}}{p \Rightarrow p}\ \Rightarrow\text{-intro}^{[1]}$$

`[1] p` = boxed assumption. `[=> intro [1]]` discharges it.

$\vee$-elim / case analysis Proof Trees

You write:

```
PROOF:
  r [lor elim]
    p lor q
    :: p => r
      [1] p
        r [from p]
    :: q => r
      [2] q
        r [from q]
```

You get:

$$\frac{p \vee q \qquad \dfrac{\ulcorner p \urcorner^{[1]}}{r} \qquad \dfrac{\ulcorner q \urcorner^{[2]}}{r}}{r}\ \vee\text{-elim}$$

`::` = sibling premises (case arms).

Equality Chain Induction

You write:

```
EQUAL:
0 + 0
0       [0+0 = 0]
length(<>)
        [length(<>) = 0]
```

You get:

$$\begin{aligned} & 0 + 0 \\ =\ & 0 && [0{+}0 = 0] \\ =\ & \mathit{length}(\langle\rangle) && [\mathrm{length}(\langle\rangle) = 0] \end{aligned}$$

`EQUAL:` = expression chains (=). Use for $\mathbb{N}$/sets/sequences.

txt2tex Relational Databases — By Example

DAT course notation: primary keys, relational algebra, bindings, GROUP/UNGROUP.

Primary Keys

Mark a primary-key attribute with `pk` in a named schema body. The PK annotation sits *outside* the Z box so fuzz can type-check the schema cleanly.

You write:

```
schema Class
  pk class : ClassName
  country  : CountryName
  bore     : N
end
```

You get:

Class

$$\begin{array}{l} class : ClassName \\ country : CountryName \\ bore : \mathbb{N} \end{array}$$

$$\mathrm{PK}(\mathrm{Class}) = \{class\}$$

Composite: two `pk` lines $\Rightarrow \mathrm{PK}(S) = \{a, b\}$. Comma-separated names on one `pk` line also work. `pk` is rejected in `axdef` / `gendef` / inline schema text.

Relational Algebra

`sigma[p](R)`	$\sigma_p(R)$ — restrict
`pi[A,B](R)`	$\pi_{A,B}(R)$ — project
`rho[A as B](R)`	$\rho_{A \to B}(R)$ — rename
`R bowtie S`	$R \bowtie S$ — natural join
`R bowtie [p] S`	$R \bowtie_p S$ — theta-join
`R div S`	$R \,\mathsf{div}\, S$ — division
`T := R`	$T := R$ (zed block)

`sigma`/`pi`/`rho` bind at atom level (prefix + args). `bowtie` and `div` are infix at cross-product precedence.

Fuzz note: algebra expressions emit *outside* any Z environment (as `\noindent$...$`), so fuzz silently skips them — your schemas and axdefs in the same document still type-check cleanly. Do *not* wrap algebra inside `zed`/`axdef`/`schema` blocks; fuzz parses block contents strictly as Z and will reject the algebra subscripts.

You write:

```
BigGuns := pi[class,country](
  sigma[bore >= 16](Class))
```

You get:

$$BigGuns := \pi_{class,country}(\sigma_{bore \geq 16}(Class))$$

Z Bindings & Set Comprehension

Z RM §3.7 binding brackets construct labelled tuples.

`{| name == e |}`	$\langle name == e \rangle$
`{| a == e, b == f |}`	multiple components
`{| |}`	empty binding

Multi-typed comprehension with binding:

You write:

```
{ s : Ship; c : Class |
  s.class = c.class .
  {| name == s.name |} }
```

You get:

$$\{s : Ship;\ c : Class \mid s.class = c.class \bullet \langle name == s.name \rangle\}$$

Use `;` to separate variable-type pairs inside `{ }`. Binding components are **comma**-separated (Z RM §3.7).

Fuzz note: bindings emit *outside* any Z environment, so fuzz silently skips them. Schemas and axdefs in the same document still type-check. Do *not* place a binding inside a `zed`/`axdef`/`schema` block; fuzz parses block contents strictly and rejects `==` inside $\langle \ldots \rangle$ even though both the brackets and the operator are defined in `fuzz.sty`.

GROUP & UNGROUP

Date's nested-relation operators.

`R group ({A,B} as alias)`	R GROUP$(\{A, B\}$ AS $alias)$
`R ungroup alias`	R UNGROUP $alias$

You write:

```
Members group ({name,email} as contact)
Members ungroup contact
```

You get:

$$Members \text{ GROUP } (\{name, email\} \text{ AS } contact)$$
$$Members \text{ UNGROUP } contact$$

Fuzz note: GROUP/UNGROUP emit *outside* any Z environment, so fuzz silently skips them. Do *not* place these inside `zed`/`axdef`/`schema` blocks — the `\mathop` wrapping and Date braces sit outside Z's grammar and fuzz rejects them in block contents.

You get:

`group`/`ungroup` use `\mathop{\mathrm{...}}` — kernel LaTeX, no extra package.

DAT Quick Reference

`pk a :  T`	primary key in schema
`pk a, b :  T`	composite pk
`sigma[p](R)`	restrict
`pi[A](R)`	project
`rho[A as B](R)`	rename attr
`R bowtie S`	natural join
`R div S`	division
`T := expr`	algebra assignment
`{| a == e |}`	binding bracket
`{S; C | P . E}`	multi-typed set comp
`R group ({A} as x)`	group attrs
`R ungroup x`	ungroup attr